MergeLens: Predicting Pull Request Outcomes from Titles and Descriptions

Concept Paper: Draft, November 2026

Timo Gräf, Maria Lyoteva, Dominik Schwagerl, and Tsvetan Stanchev
Department of Electrical Engineering, Media, and Computer Science
Ostbayerische Technische Hochschule Amberg-Weiden
Amberg, Germany

***Abstract*—Maintainers of active GitHub repositories must assess numerous pull requests (PRs), including proposals of uncertain value. *MergeLens* investigates whether a compact, fine-tuned language model can provide an early indication of a PR's potential usefulness using only its English title and description. Historical GH Archive events will supply labeled examples based on observed merge outcomes. The proposed system comprises a reproducible data-processing and training pipeline, a containerized inference service, and a web interface for maintainer-triggered predictions. Its predictive value will be assessed on held-out data; it is intended to support, not replace, human review.**

***Index Terms*—pull requests, GitHub, GH Archive, text classification, fine-tuning, maintainer support.**

I. Introduction and Motivation

Popular open-source repositories may receive more pull requests (PRs) than their maintainers can promptly review. Each proposal requires an assessment of its relevance, technical contribution, and fit with the project's goals. AI-assisted coding tools can make it easier to submit contributions, including superficially plausible proposals that offer limited value. This increases the importance of efficient initial triage, particularly when maintainers have limited time [1].

A comprehensive code-based assessment can be costly: inspecting large changes and their repository context may require substantial input context and computational resources. Generic language models are also not specifically trained to estimate PR acceptance. *MergeLens* therefore explores a deliberately narrower approach: can the language of a PR title and description alone provide a useful first indication of whether its proposed change is likely to benefit a project? A potentially useful PR is understood here as a contribution that fixes a problem, improves an existing technical property, or introduces a relevant functional enhancement.

The system will learn from historical GitHub PRs available through GH Archive [2], using a merge as a positive training label and closure without merging as a negative label. This large-scale approximation is a practical learning signal, not a definitive measure of technical merit. The objective is to evaluate whether a relatively compact, specialized model can help maintainers prioritize manual reviews without analyzing source code or making acceptance decisions on their behalf.

II. Related Work

Chen et al.'s *E-PRedictor* predicts PR acceptance at submission time by combining BERT-derived representations of PR descriptions and code changes with contributor, PR, and project features [3]. Similarly, *CTCPPre* uses textual, code, contributor, and project characteristics to predict acceptance [4]. Both demonstrate the established relevance of acceptance prediction but rely on information beyond PR text. Pirouzkhah et al. found associations between specific elements of PR descriptions and review outcomes, including merge decisions [5]. Building on this research, the proposed experiment isolates the predictive information in the *title and description* and studies its usefulness with a fine-tuned, lightweight classifier rather than a feature-rich, code-aware system.

III. Functional Design

The architecture in Fig. 1 separates an offline learning pipeline from the online maintainer workflow. A data engineer prepares labeled historical examples; a model developer trains and evaluates the classifier; a repository maintainer views a PR retrieved via GitHub API or webhooks [6] and requests an indication of its likely merge outcome. The database retains both extracted records and processed datasets. Optional account-based navigation and contributor metadata are outside the initial pipeline.

IV. Functional Requirements: User Stories

The following user stories define the minimum viable product (MUST), desirable enhancements (SHOULD), optional extensions (COULD), and explicit exclusions (WON'T). A PR's *outcome* refers to its historically observed merged or closed-without-merge status.

A. MUST

US1 **Acquire and Label PRs.** As a data engineer, I want to extract PR-related events from GH Archive so that historical training examples can be labeled.

- **Acceptance:** Given archive files for a selected period, when extraction runs, then PR identifiers, titles, descriptions, and determinable terminal outcomes are collected; unrelated events are excluded and repeated events are associated with the same PR.

This work is licensed under a Creative Commons Attribution 4.0 International (CC BY 4.0) license.

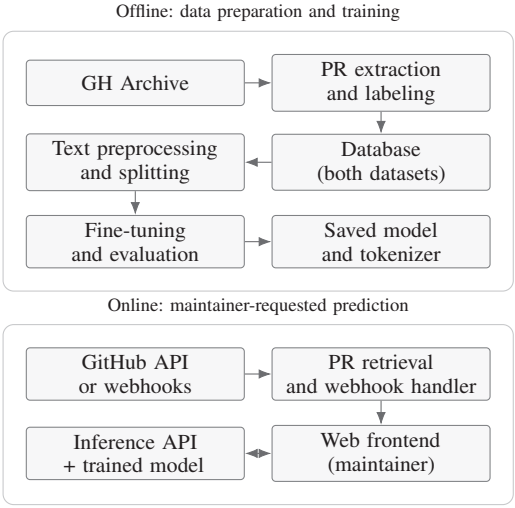


Figure 1. Proposed offline training and online PR assessment. Preprocessed datasets are written back to the database; the saved model is loaded by the inference API.

US2 **Persist Project Data.** As a data engineer, I want extracted and prepared PR records stored in a database so that downstream steps can retrieve them reliably.
- **Acceptance:** Given an extracted or processed record, when it is stored, then it can be retrieved for the corresponding next processing stage.

US3 **Prepare and Split Data.** As a model developer, I want standardized English PR text and separate datasets so that training and evaluation use consistent inputs.
- **Acceptance:** Given stored records, when preprocessing completes, then unusable records are filtered, text is normalized, English entries are selected, and train/validation/test sets are persisted without PR overlap.

US4 **Fine-Tune a Binary Classifier.** As a model developer, I want to adapt a pretrained language model so that it can predict PR merge outcomes from title and description alone.
- **Acceptance:** Given the training split, when training completes, then the model uses a two-output classification head and produces a saved model artifact and tokenizer for inference.

US5 **Evaluate Generalization.** As a model developer, I want performance measured on unseen PRs so that the value of the text-only approach can be assessed.
- **Acceptance:** Given nonoverlapping validation and test sets, when evaluation runs, then per-class precision, recall, F1, and a confusion matrix are reported; the test set is not used for training.

US6 **Request and View PR Assessment.** As a maintainer, I want to view a PR retrieved through the GitHub API or a webhook and trigger a prediction so that I can prioritize my review.
- **Acceptance:** Given a retrievable PR, when I open the view, then its title and description are displayed; when I select *Evaluate*, the frontend calls the inference API and displays a prediction or an actionable error.

US7 **Run Integrated Modules.** As a developer, I want the project's modules containerized so that the data workflows, database, model service, and frontend can be integrated reproducibly.
- **Acceptance:** Given the documented Docker configuration, when the required services are started, then the components connect and a sample PR evaluation reaches the inference service.

B. SHOULD

US8 **Display a Prediction Score.** As a maintainer, I want a clearly labeled model-derived score alongside the predicted class so that I can judge the strength of the initial indication.
- **Acceptance:** Given a successful inference, the UI shows the predicted class and a clearly labeled score without presenting it as proof of PR quality.

C. COULD

US9 **Browse PRs Through GitHub Login.** As a maintainer, I want to sign in with GitHub, list open PRs for an accessible repository, and select one for evaluation.
- **Acceptance:** After authorization, selecting a listed PR displays its title and description and enables the same evaluation action as in US6.

US10 **Compare Additional Contributor Features.** As a model developer, I want to include contributor metadata, such as account age or commit history, so that I can test whether it improves predictive performance.
- **Acceptance:** If implemented, the acquisition, preprocessing, and model pipelines support the added features, and evaluation compares them with the title-and-description baseline.

D. WON'T (MVP)

The system will neither autonomously accept/reject PRs nor inspect complete code changes for its initial prediction. Mandatory cloud hosting is also outside the MVP scope.

V. Non-Functional Requirements

Scalability and reliability: Historical archives should be processed incrementally without loading the entire corpus into memory; failed jobs should be restartable without duplicating PR records. **Reproducibility:** Dataset splits, preprocessing choices, model settings, and evaluation outputs should be traceable to a run and model version. **Usability and performance:** Predictions and failures must be presented clearly; inference latency should be measured on the target local hardware. **Security:** The MVP uses publicly accessible PR data; any future GitHub authentication must protect access tokens and request only necessary permissions.

VI. Technical Key Components and Technology Stack

The proposed pipeline uses **Python** for GH Archive ingestion and PR extraction. **Polars** [7] is a candidate for memory-efficient, single-machine batch processing; **PySpark** [8] remains an option if the selected observation period and available infrastructure justify distributed execution. A **PostgreSQL** database [9] is under consideration for structured PR records and both raw-extracted and prepared datasets; its schema is not yet fixed.

Model development will likely use **PyTorch** [10] and **Hugging Face Transformers** [11]. A compact **BERT-based encoder** [12] with a two-logit classification head is a candidate, not a final selection. **scikit-learn** [13] may support metrics and a simple baseline. For deployment, a **FastAPI** service [14] will load the saved model and tokenizer and expose inference to a prospective **React/TypeScript** frontend [15, 16]. **Docker** [17], potentially coordinated with **Docker Compose** [18], will support local execution; a public cloud service is not required. Basic CI with **GitHub Actions** [19] may be considered.

The main design trade-off is lower inference input size and simpler integration versus missing code and repository context. Practical risks include variable archive payloads, class imbalance, labeling noise, GitHub API limits, and fine-tuning cost. The training texts should reflect information available when a PR is submitted, avoiding outcome-related leakage; a simple text baseline may be used for comparison. Observation period, model size, compute budget, and final deployment configuration will be chosen during implementation.

VII. Conclusion and Outlook

MergeLens proposes a text-only, early PR triage indicator built from historical GitHub outcomes. Its central experiment is whether title and description carry enough signal for useful predictions with a compact fine-tuned model. A separate test set will establish the approach's limitations before any claim of practical benefit. Optional GitHub sign-in and contributor metadata may broaden usability and predictive context if resources permit; maintainer judgment remains authoritative.

References

[1] Sebastian Baltes, Marc Cheong, and Christoph Treude. *An Endless Stream of AI Slop: How Developers Discuss the Burden of AI-Assisted Software Development*. 2026. DOI: 10.48550/arXiv.2603.27249. arXiv: 2603.27249.

[2] GH Archive. *GH Archive: Public GitHub Activity Archive*. URL: https://www.gharchive.org/ (visited on 11/08/2026).

[3] Kexing Chen et al. "E-PRedictor: An Approach for Early Prediction of Pull Request Acceptance". In: *SCIENCE CHINA Information Sciences* 68.5, 152104 (2025). DOI: 10.1007/s11432-022-3953-4.

[4] Jing Jiang, Jia-teng Zheng, Yun Yang, and Li Zhang. "CTCPPre: A Prediction Method for Accepted Pull Requests in GitHub". In: *Journal of Central South University* 27.2 (2020), pp. 449–468. DOI: 10.1007/s11771-020-4308-z.

[5] Shirin Pirouzkhah, Pavlina Wurzel Gonçalves, and Alberto Bacchelli. "The Value of Effective Pull Request Description". In: *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR '26)*. 2026. DOI: 10.1145/3793302.3793368.

[6] GitHub. *REST API Endpoints for Pull Requests*. URL: https://docs.github.com/en/rest/pulls/pulls (visited on 11/08/2026).

[7] Polars Contributors. *Polars User Guide*. URL: https://docs.pola.rs/ (visited on 11/08/2026).

[8] Matei Zaharia et al. "Apache Spark: A Unified Engine for Big Data Processing". In: *Communications of the ACM* 59.11 (2016), pp. 56–65. DOI: 10.1145/2934664.

[9] PostgreSQL Global Development Group. *PostgreSQL Documentation*. URL: https://www.postgresql.org/docs/current/ (visited on 11/08/2026).

[10] Adam Paszke et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019, pp. 8026–8037. URL: https://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.

[11] Thomas Wolf et al. "Transformers: State-of-the-Art Natural Language Processing". In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 2020, pp. 38–45. DOI: 10.18653/v1/2020.emnlp-demos.6.

[12] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423.

[13] Fabian Pedregosa et al. "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830. URL: https://www.jmlr.org/papers/v12/pedregosa11a.html.

[14] Sebastián Ramírez. *FastAPI Documentation*. URL: https://fastapi.tiangolo.com/ (visited on 11/08/2026).

[15] React Team. *React Documentation*. URL: https://react.dev/ (visited on 11/08/2026).

[16] Microsoft. *The TypeScript Handbook*. URL: https://www.typescriptlang.org/docs/handbook/ (visited on 11/08/2026).

[17] Dirk Merkel. "Docker: Lightweight Linux Containers for Consistent Development and Deployment". In: *Linux Journal* 239 (2014). URL: https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment.

[18] Docker Inc. *Docker Compose Documentation*. URL: https://docs.docker.com/compose/ (visited on 11/08/2026).

[19] GitHub. *GitHub Actions Documentation*. URL: https://docs.github.com/en/actions (visited on 11/08/2026).